Meta-Learning for 1D-ARC

Assignment 1

Shamik Sinha | Roll No. 2022468

CSE618 : Meta Learning, Winter Semester 2026
IIIT Delhi

1D Abstraction and Reasoning Corpus (1D-ARC)

Given 2–4 (input, output) example pairs for an unknown transformation rule, infer the rule and apply it to a new test input.

Dataset

- 18 concept classes: copy, flip, move, fill, denoising, recolor, scale, and more
- 50 tasks per concept; 901 tasks total
- Tokens from $\{0, 1, \ldots, 9\}$; token 0 is background

Evaluation Settings and Preprocessing

- **Same-concept:** 80/20 split within each concept class
- **Cross-concept:** 4 entire concept classes held out; model must generalise to unseen transformation types
- **Data Augmentation:** Token colour remapping was used to augment existing data

Why Meta-Learning?

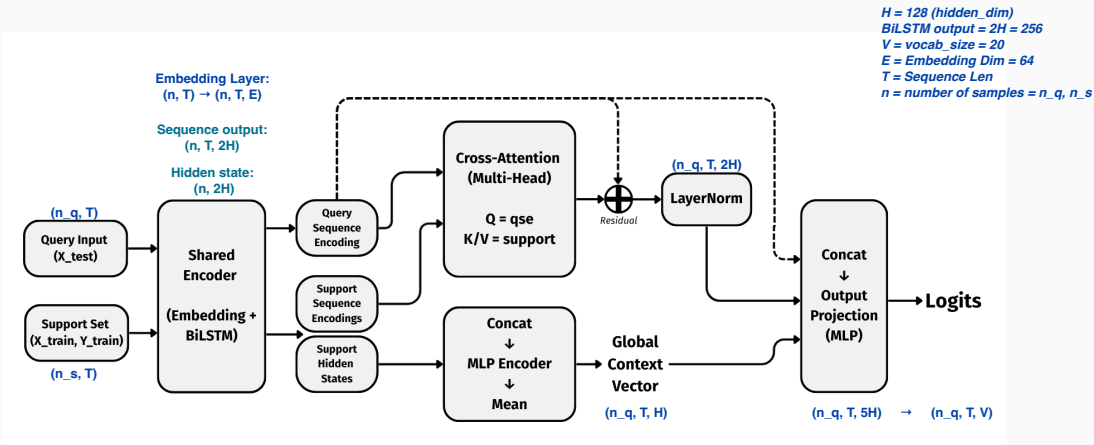
Each task has a different rule. A model must infer the rule from 2–4 examples on the spot. Standard supervised training struggles to generalise across tasks with very few examples; meta-learning trains the model to **learn how to learn**.

Models Used

I used the following models in this assignment:

#	Model	Key Idea	Why i used it
1	BiLSTM baseline	Simple supervised model (predict output directly)	Set a basic performance baseline
2	BiLSTM + FOMAML	Learns how to adapt using gradients	Helps the model improve using few examples at test time
3	Matching Networks	Uses attention over support examples	Learns to compare with support examples
4	PosCNP	Adds cross-attention to get better positional information	Helps the model understand where things are in the sequence

Model Architecture of the Best Model (PosCNP)



Shared BiLSTM encoder processes both query and support sequences. Cross-attention (Q = query encoding, K/V = support encodings) recovers per-position transformation signal. A global context vector from mean-pooled support is concatenated before the output MLP.

Training Methods and Loss Functions

#	Model	Training Method	Key Detail
1	BiLSTM baseline	Standard supervised training on shuffled minibatches	No support set; no test time adaptation (TTA); grad clip 1.0
2	BiLSTM + FOMAML	First-order MAML with inner-loop adaptation on support tasks	GD on support in inner loop; meta-update using query loss; grad clip 1.0
3	Matching Networks	Tasks shuffled; one task per epoch	Support set as context; no TTA; grad clip 1.0
4	PosCNP	Tasks shuffled; one task per epoch (same as Matching Networks)	Support set as context; no TTA; cross-attention
5	PosCNP + FOMAML	First-order MAML with episodic tasks (cross-concept setting)	GD on support st in inner loop; meta-update using query loss; uses support as context; grad clip 1.0

† All models use pixel-wise **cross-entropy loss** with padding tokens ignored.

Evalutaion and Results

I used exact match accuracies for both same- and cross-concept evaluation settings.

Same-Concept Accuracy (%)

Model	Orig	+Adapt	+Aug
BiLSTM baseline	34.8	–	43.5
BiLSTM + FOMAML	13.3	29.8	61.3
Matching Networks	55.8	–	75.7
PosCNP	69.6	–	**78.5**

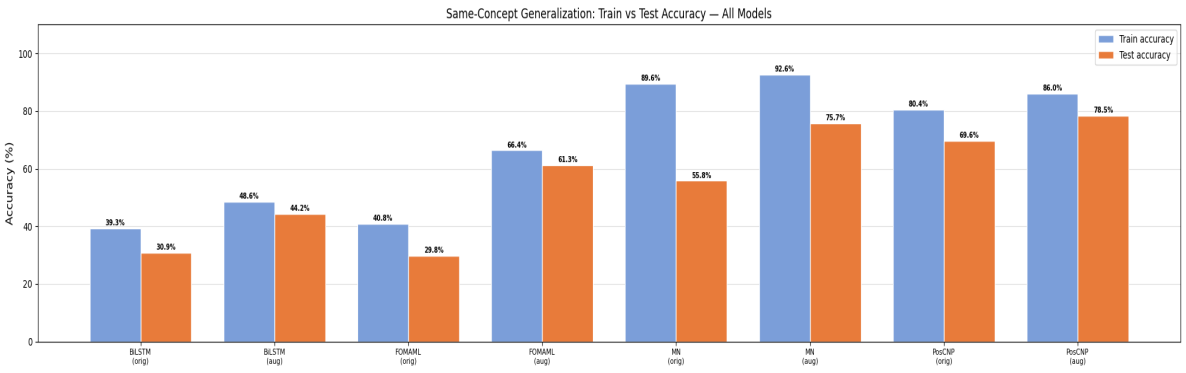
Cross-Concept Accuracy (%)

Model	Orig	+Adapt	+Aug
BiLSTM baseline	27.9	–	25.8
BiLSTM + FOMAML	16.0	43.0	38.5
PosCNP + MAML	34.5	50.5	**60.5**

Held-out test concepts: `fill`, `hollow`, `move_2p`, `move_dp`

Orig = no adaptation. +Adapt = 20 SGD steps on support set at test time (original data). +Aug = augmented training data; FOMAML and PosCNP+MAML still use 20 adapt steps.

Analysis: Overfitting in Same-Concept Setting



MN overfit significantly

MN (orig): **89.6%** train vs **55.8%** test, a 34-point gap.
MN (aug): 92.6% vs 75.7%, still large.

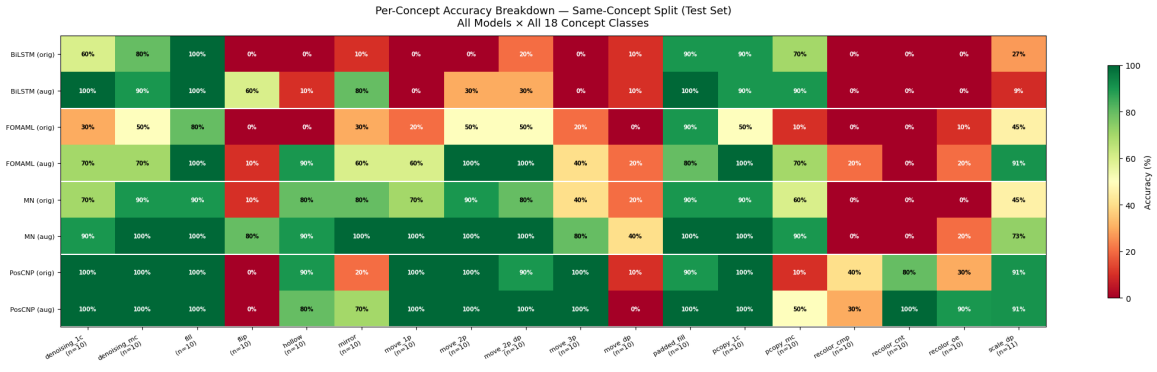
Other models less affected

BiLSTM and FOMAML have smaller train-test gaps. PosCNP (orig): 80.4% vs 69.6%, reasonable.

Augmentation/Regularisation helped

Data augmentation reduced gaps across all models. Dropout added to PosCNP to directly regularise.

Analysis: Per-Concept Breakdown and Cross-Concept Robustness



60.5% is misleading

Depends entirely on *which* 4 concepts are held out. A different split gives ∼3% to ∼97%.

Some tasks inherently harder

Recolor tasks are all red (0–9%) even in same-concept. No model has a counting or color-mapping inductive bias.

PosCNP: tradeoff

Improved recolor (10–40% from 0%) but slightly hurt previously easy concepts. Attention redistributes capacity.

Towards a More Robust Evaluation Strategy for Cross-Concept

The Problem: A single fixed split is unreliable. The same model scores anywhere from ∼3% to ∼97% depending only on *which* 4 concepts are held out.

Strategy 1: Random-Split Averaging *implemented*

Train a fresh PosCNP+MAML per round on 14 randomly sampled concepts; test on the remaining 4.

Round (held-out concepts)	Acc
R1: denoising_1c, flip, move_2p_dp, scale_dp	32.8%
R2: denoising_1c, fill, move_3p, recolor_oe	17.0%
R3: hollow, move_2p_dp, move_3p, padded_fill	15.5%
R4: denoising_1c, denoising_mc, recolor_cmp, recolor_cnt	8.0%
R5: mirror, padded_fill, pcopy_1c, recolor_cmp	3.0%
Mean ± std	**15.3% ± 10.1%**

Compute Constraint

Each PosCNP+MAML run takes **upwards of 1 hour**, so:

- Only **5 epochs** per round (vs. full 15)
- Only **5 splits** (vs. ideal 10+)

The 15.3% mean is therefore a *lower bound*, full training would likely be higher.

Strategy 2: Leave One Out Strategy *not run*

Train **18 separate models**, each leaving out exactly one concept class; every concept appears as test exactly once.

At ∼1 hour per model this requires **18+ hours** of uninterrupted GPU time. This remains the ideal evaluation approach.

Thank You!